Protobuf to Cap'n Proto in `f1r3node-rust`:

where the performance is, and what a port would cost

F1R3FLY.io

20 October 2026 (revised) · `dev` @ `e87572d`

Abstract

This note asks whether replacing Protocol Buffers (prost/tonic) with Cap'n Proto in `f1r3node-rust` would yield worthwhile performance gains, and what such a port would cost. `f1r3node-rust` is a platform: organisations stand up their own shards with it, from the Artificial Superintelligence Alliance's ASI:Chain to internal deployments, and shards are cheap to create. An encoding change is therefore a *shard-format release* that each deployment adopts on its own schedule, not a fork of one shared history. The answer has three parts. First, the protobuf encoding is more than a wire format: it is the input to the block hash, deploy signatures, unforgeable-name seeds, registry signatures, address derivation and Rholang's `toByteArray`. A port has to choose a canonical hashing encoding for all of these and update every client SDK's deploy signing. Second, microbenchmarks on the node's own message shapes show Cap'n Proto encodes 1.4–2.4× faster (except for small terms) and reads in place 3–5× faster, but materialising owned Rust structures from Cap'n Proto is *slower* than prost (1.1–2×), its canonical form for hashing is 3.6–7× slower, and unpacked messages are 43% (blocks) to nearly 7× (Rholang terms) larger. Third, the whole protobuf codec bill for a typical 32-deploy block is about 1 ms per validator, against roughly 320 ms of deploy evaluation; the best realistic Cap'n Proto port recovers about 0.5 ms of it. The recommendation is not to port, because the gain is too small for the work. The prost-side fixes in Section 8 recover more for a few lines of code. Since hash formats can be changed per release, a Merkle-root block hash is the better use of that freedom: it takes the end-of-block hash check from 463 to 7 µs and gives per-deploy inclusion proofs. Serialisation effort beyond that belongs on RSpace's bincode hashing, which runs on every produce and consume.

1 Scope and method

The analysis covers the `dev` branch of `F1R3FLY-io/f1r3node-rust` at commit `e87572d` (merge of PR #662, 20 October 2026). It has three parts.

1. An inventory of every protobuf schema and every encode/decode call site, classified by the role the bytes play (wire, storage, hash input, in-memory type, external API).
2. A standalone benchmark crate that compiles the repository's real `CasperMessage.proto` and `RhoTypes.proto` with the same prost-build settings the node uses (`btree_map(".")`, `Bytes` for `.casper/.routing`), alongside a Cap'n Proto schema mirroring the same fields. Round-trip equality is asserted for every workload before timing.
3. A per-block budget combining the benchmark numbers with the call paths found in (1) and the node's own propose-latency measurements.

Deployment context. The node is software that other organisations run, each as one or more shards: ASI:Chain is one such deployment and several companies run internal ones. Shards are inexpensive to stand up and are separated by `networkId`. A change to a hashed or signed encoding is therefore a new shard format shipped in a release. Existing shards keep running the release they are on until their operators move; new shards start on the new format. The costs that remain are engineering and ecosystem costs (client SDKs, tooling, contracts that depend on byte formats), which this note estimates, rather than the preservation of a single history.

The node itself was not built: its pinned `nightly-2026-02-09` toolchain could not be fetched in the analysis environment. Section 9 lists the consequences.

2 Where protobuf lives in the node

The repository carries 13 schema files (2 022 lines, 214 messages, 30 RPCs). Generated code comes from `tonic-prost-build` in `models`, `comm` and `node`. Table 1 groups the uses by the role of the bytes.

Role	What	Where (representative)
P2P wire	`TransportLayer` gRPC (unary `Send`, client-streaming `Stream`); `Protocol→Packet{typeId, content}` with the Casper message encoded inside (double encoding); 256 KiB chunks; LZ4 above 500 KB; rustls with node-id from cert.	`routing.proto:62`, `packet_type_tag.rs:29`, `chunker.rs:15`, `casper/src/rust/protocol/mod.rs:273`
Block storage	LMDB (heed). `blocks`: LZ4(prost(`BlockMessageProto`)); `blocks-approved`: prost; deploy buffers: tagged prost. DAG indices, latest messages, casper buffer, finality: bincode.	`key_value_block_store.rs:263`, `deploy_codec.rs:24`, `block_dag_key_value_storage.rs:974`
Hash and signature input	Block hash, deploy signature, genesis approval, unforgeable seeds, registry signatures, `toByteArray`, unforgeable-to-address (Sec. 2.1).	`proto_util.rs:467`, `signed.rs:33`, `mergeable_tags.rs:55`
In-memory types	The generated `rhoapi` structs (`Par`, `Send`, `Expr`, ...) *are* the interpreter's AST, with serde, `Ord` and utoipa derives added in `build.rs`. 542 import sites; 146 files in `rholang`, 63 in `casper`.	`models/build.rs:43-54`
External API	gRPC on 40401/40402: Deploy (19 RPCs), Propose, Repl, LSP, plus tonic-reflection; HTTP/JSON on 40403 via serde on the `rhoapi` types; clients include pyf1r3fly, system-integration, Embers, grpcurl.	`grpc_package.rs:117`, `web_api_routes.rs:40`

Table 1: Roles of protobuf bytes and types in `f1r3node-rust` (`dev`). Paths abbreviated.

The serialiser that does the most work during block execution is not protobuf. RSpace hashes every channel, datum and continuation by `bincode::serialize` followed by Blake2b256 (`stable_hash_provider.rs:7`), and trie leaves are bincode too (`history_repository_impl.rs:226-330`). That path runs on every produce and consume.

2.1 Encodings that feed hashes and signatures

Each of the following hashes or signs prost output. Within a shard, every node must produce the same bytes, so each of these is part of the shard format.

1. **Block hash.** `hash_block` concatenates prost(`Header`), prost(`Body`) and raw fields, then Blake2b256 (`casper/src/rust/util/proto_util.rs:467-484`); re-checked on every received block (`validate.rs:1179`).
2. **Deploy signatures.** Wallets and clients sign Blake2b256 (or Keccak256 for eth-style keys) of prost(`DeployDataProto`) with `sig/deployer` cleared (`crypto/src/rust/signatures/signed.rs:33`; `casper_message.rs:1383`).
3. **Genesis ceremony.** Approvals sign Blake2b256 of prost(`ApprovedBlockCandidate`) (`validate.rs:167`); genesis deploy-log order is sorted by prost bytes (`genesis.rs:249`).
4. **Unforgeable names.** The mergeable-tag RNG is seeded from prost(`DeployDataProto{deployer, timestamp}`) (`rholang/.../merging/mergeable_tags.rs:49-56`).
5. **Addresses and registry.** Unforgeable-to-address is Keccak256(prost(`GPrivate`)) (`address_tools.rs:82`); registry signatures are Blake2b256(prost(`Par`)) and are baked into the genesis contracts a shard ships with (`registry_sig_gen.rs:214`).
6. **Language semantics.** Rholang's `toByteArray` returns prost(`Par`) (`reduce.rs:3181`); contracts can store or hash the result.

Consequence

A port can replace prost entirely in a new shard format, so no permanent dual codec is forced. It must still (i) pick one *canonical* byte form for each of these six families, (ii) ship new deploy-signing code in every client SDK (pyf1r3fly, Embers, partner wallets), (iii) regenerate the signed genesis contracts, and (iv) document that `toByteArray` returns different bytes on the new format. Section 3 shows Cap'n Proto is a poor choice for (i), so a realistic port uses Cap'n Proto on the wire and in storage and keeps a separate, simpler hashing encoding.

3 Benchmarks

3.1 Setup

Workloads come from deterministic generators. Blocks follow `BlockMessageProto`: 5 validators (bonds, justifications), 3 parents, and N processed deploys, each carrying a 1.1 KB Rholang transfer term, 65-byte deployer key, 71-byte signature, and E deploy-log events (uniform mix of produce, consume with 1–3 channel hashes, and comm). Terms (`Par`) are random trees of sends, receives, `news`, tuples and leaf expressions at depths 2, 4 and 6. The Cap'n Proto side builds directly from the prost structs (standing in for domain types) and reads back into them, which is what the node's `to_proto`/`from_proto` layer would become.

Hardware: 2-vCPU Intel Xeon @ 2.8 GHz VM, Rust stable, `opt-level=3`, thin LTO, `codegen-units=1`; prost 0.14.4, capnp 0.27.2. Each figure is the median of 11 batches of ≥40 ms. Run-to-run variation on this shared VM was up to ±15%; ratios were stable.

3.2 Results

	encode		decode → owned			read all fields		sigs only
Workload (size)	prost	capnp	prost	prost `Bytes`	capnp	prost	capnp	capnp
block 1×40 (7 KB)	5.8	4.1	10.8	8.6	16.0	11.1	3.1	0.1
block 32×40 (207 KB)	202	145	423	278	550	422	152	1.3
block 32×400 (1.7 MB)	2 483	1 585	4 274	2 552	5 778	4 483	949	1.7
block 200×40 (1.3 MB)	1 548	972	3 036	1 906	3 338	3 379	680	7.9

Table 2: Block codec costs in µs (median). "prost `Bytes`" decodes from a `bytes::Bytes` buffer so that `bytes` fields are reference-counted slices instead of copies. "read all" is decode-then-walk for prost and an in-place walk for Cap'n Proto. "sigs only" reads every deploy signature; prost must decode the whole block for that (≈ its decode column). Small blocks are shown to one decimal place.

	encode			decode → owned		hash input + Blake2b	
`Par` (prost size)	prost	capnp	bincode	prost	capnp	prost	capnp canonical
small (262 B)	1.3	1.7	0.6	2.3	4.5	1.8	12.3
medium (4.6 KB)	45.9	25.2	11.7	43.8	81.6	60.0	365.7
large (28 KB)	458	191	73	332	617	498	1 840

Table 3: Rholang term (`Par`) codec costs in µs. bincode is the RSpace hashing codec; bincode+Blake2b for the same terms took 2.7, 59.7 and 275 µs.

3.3 Reading the numbers

Encoding

Cap'n Proto building is 1.4–1.6× faster on blocks and 1.8–2.4× faster on medium and large terms (slower on small ones). The term result is partly a prost artefact: protobuf length-prefixes every nested message and prost recomputes nested sizes at each level, so encode cost grows with depth×size (28 KB of `Par` costs more than twice as much to encode as a 207 KB block).

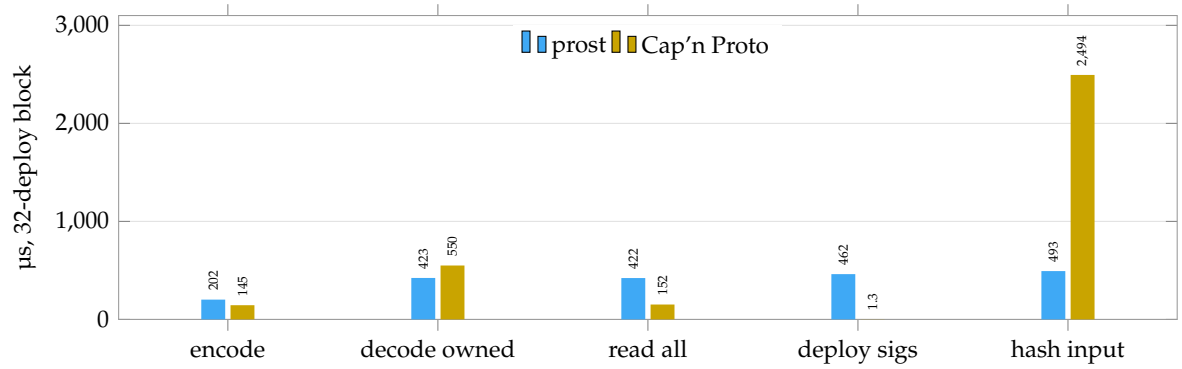


Figure 1: The 32-deploy, 40-events-per-deploy block. Cap'n Proto wins wherever the consumer can stay on the reader; it loses wherever an owned structure or a canonical hash input is needed, which is what the node's current architecture asks for.

	prost		capnp		capnp packed	
Workload	raw	+LZ4	raw	+LZ4	raw	+LZ4
block 32×40	206 607	176 846	294 960	186 953	234 038	180 209
block 200×40	1 281 438	1 093 251	1 820 304	1 154 215	1 449 250	1 112 395
`Par` medium	4 638	3 379	32 072	9 962	13 687	5 942
`Par` large	28 229	19 438	188 968	41 896	81 525	31 421

Table 4: Encoded sizes in bytes. Packed Cap'n Proto closes most of the gap but its encode (268 µs on the 32-deploy block) and decode (661 µs) are then slower than prost.

Owned decode

Building owned Rust values from a Cap'n Proto message costs more than prost's decode, because the reader still validates every pointer and the allocations are the same. Decoding prost from `Bytes` widens the gap: 278 vs. 550 µs.

In-place reads

This is the Cap'n Proto advantage: a full walk is 3–5× cheaper than a prost decode, and selective access (deploy signatures) is three orders of magnitude cheaper. It is only available to code that consumes readers rather than owned structs.

Hash inputs

Cap'n Proto admits many layouts for one value, so a hash needs `canonicalize()`: 3.6–7× the cost of prost encode for the same Blake2b. This rules Cap'n Proto out as the canonical *hashing* format of a new shard release, even where it carries the wire and storage traffic (Sec. 2.1).

Size

Unpacked messages are 43% larger for blocks and nearly 7× larger for terms (8-byte alignment, pointer words, no varints). Every block is gossiped to every peer, so this is bandwidth, not just memory.

4 The per-block budget

Table 5 traces a 32-deploy block through a receiving validator using the call paths in the code and the timings above. Clones from `to_proto`/`from_proto` are estimated by a struct clone of the same data. Protobuf encode/decode is therefore about 0.3% of block handling, and everything serialisation-related (including Blake2b and LZ4, which no codec choice removes) about 0.7%. A Cap'n Proto port that keeps the domain layer and a prost-style hashing encoding would change this column as follows: decode +127; size metric −373 (the message knows its size); store encode −228; hash check unchanged. Net saving ≈0.5 ms per block, about 0.15% of block time. A deeper port that rewrites `casper` to consume readers directly (dropping `from_proto`) might reach ≈0.8 ms. The prost-side fixes in Section 8 recover ≈0.9 ms with a handful of line changes, and the Merkle-root hash of Section 6 removes a further ≈0.6 ms from the end-of-block critical path.

Step	Code	µs
Decode `BlockMessageProto` from `&[u8]`	`protocol/mod.rs:275`	423
`from_proto` into domain `BlockMessage`	`casper_message.rs`	≈171
Size metric: `to_proto` + full encode, length only	`block_processor.rs:542`	373
Hash check: `to_proto` + prost(header,body) + Blake2b	`validate.rs:1179`	664
Store: `to_proto` + encode + LZ4	`key_value_block_store.rs:263`	574
Total serialisation-related		≈2 200
of which prost encode/decode		≈1 030

Table 5: Serialisation work per received 32-deploy block. For scale, the node's recorded propose profile shows `checkpoint_compute_state` averaging 319 ms (p95 460 ms) and `propose_total` 312 ms (`docs/archive/adaptive-deploy-cap-validation-2026-03-05.md`).

5 What a port would cost

A new shard format

Because deployments adopt releases independently, a port can replace prost outright in a new format. The release must still define a canonical hashing encoding for the six families of Section 2.1, regenerate signed genesis contracts, and publish golden test vectors so that partner implementations agree byte-for-byte.

The Rholang AST

capnp-rust generates readers and builders, not owned structs. The interpreter, normaliser, sorter, matcher and RSpace all operate on owned `rhoapi` values (with serde for bincode hashing, `Ord`, utoipa). Those structs stay; Cap'n Proto becomes one more conversion layer for the 58 term messages, hand-written or via a third-party derive.

Client SDKs

Deploy signing happens in client code. Every SDK (pyf1r3fly, Embers, partner wallets such as ASI:Chain's) needs a signer for the new format, and must know which format a given shard speaks.

Format identification

The handshake carries no protocol or format version; the only gate is `networkId` (`rp/connect.rs:450`). Nodes of one shard upgrade together, so no mixed-format shard is needed, but a format identifier in the handshake (and in the deploy API) would make a mismatched node or client fail with a clear error instead of a decode error.

RPC and tooling

Cap'n Proto RPC is not gRPC. The Rust `capnp-rpc` crate is built around a single-threaded event loop (its examples run under Tokio's `LocalSet/spawn_local`), whereas `comm` is built on multi-threaded tonic with custom rustls interceptors, chunked streaming and circuit breakers. The practical route is a custom tonic `Codec` carrying Cap'n Proto bytes, which forfeits tonic-reflection and grpcurl for those services.

External APIs

pyf1r3fly, system-integration and Embers use protobuf/gRPC stubs, and the HTTP API serialises the `rhoapi` types to JSON via serde. Moving the public API means new stubs for every client and partner integration; keeping it on protobuf means that surface gains nothing.

Storage

New shards start with the new store format. A migrator is only needed for an existing deployment that wants to move its history rather than start a new shard; the `deploy_codec.rs` format tag shows the pattern.

Size and safety

43% more bytes per gossiped block unless packed (which loses the speed); readers need traversal and nesting limits tuned against the 256 MB stream ceiling.

6 Using the freedom differently: a Merkle-root block hash

Because hash formats are per-release, the block-hash input itself can be redesigned without any change of codec. Today `hash_block` encodes the whole header and body and hashes them in one serial pass at the

Scope	Main work	Eng.-weeks
A. P2P payloads only (Casper + routing)	∼70 messages to schema; conversions; tonic codec; format identifier in handshake	5–8
B. A + block/deploy storage	new store format; optional migrator for existing shards	+2–3
C. B + hashing encoding redefined	canonical spec, golden vectors, SDK signers, genesis contracts	+2–3
D. C + casper consuming readers directly	rewrite of block handling over 63+ files	+8–12
E. Public APIs and `rhoapi`	client stubs, JSON layer, interpreter types	large; little gain

Table 6: Rough effort, assuming one engineer familiar with the code base. Ranges are judgement, not measurement; the expected gain for A–D is under 0.3% of block time (Section 4).

end of validation, after cloning both through `to_proto`. The alternative hashes each processed deploy on its own (a leaf), builds a binary Merkle tree over the leaves, and hashes the header together with the root. A validator computes each leaf from its own replay result as that deploy finishes, so at the end of the block only the tree and header remain.

	current	Merkle root over deploys		
Workload	whole body	all leaves + root	root only	one leaf
block 1×40	13.5	12.4	0.5	11.6
block 32×40	463	468	7.0	11.7
block 32×400	4 733	4 618	7.1	126
block 200×40	3 263	2 918	43	13.6

Table 7: Block-hash input cost in µs, prost encoding plus Blake2b in both cases. "root only" is the end-of-block work when leaves were computed during replay.

The total hashing work is unchanged, but it moves off the critical path: for a 32-deploy block the end-of-block check falls from 463 to 7 µs, and the `to_proto` clone (≈171 µs) disappears, about 0.6 ms in all. That is as much as the Cap'n Proto port recovers, without touching the wire, storage or client APIs. The structure also brings benefits a flat hash cannot:

- a mismatching deploy is detected as soon as its replay finishes, not at the end of the block;
- leaves can be hashed in parallel across cores;
- any party can prove that a deploy and its result are in a block with $\lceil \log_2 n \rceil$ sibling hashes, which matters to partner deployments that expose results to their own users or to light clients.

Deploy signatures, unforgeable seeds and the other families of Section 2.1 are unaffected and can stay on prost.

7 Where Cap'n Proto would genuinely help

The measured strengths are in-place and selective reads. Three places in the node could use them: serving blocks to syncing peers straight from storage, look-ups that need a few fields of a stored block (deploy signatures, parents), and very deep term encodes. The first two already have cheaper answers within the current design: the block store has a bounded decoder and deploy-sig cache (`key_value_block_store.rs:328-419`) and in-memory DAG caches, and stored bytes could be forwarded to peers without a decode/re-encode. Deep term encodes happen off the hot path (`toByteArray`, registry signatures, non-deterministic system-process outputs), and where they matter for hashing, bincode already encodes them 6× faster than prost.

8 Recommendations

1. **Do not port.** Even with shard-format releases removing the need for a dual codec, the upper bound on the gain is a fraction of a percent of block time, against two to four months of engineering plus SDK and partner coordination.
2. **Decode from `Bytes`.** `parse_packet` calls `T::decode(packet.content.as_ref())` (`casper/src/rust/protocol/mod.rs:275`); passing `packet.content.clone()` makes every `bytes` field a reference-counted slice. Measured: 35–40% off block decode. Do the same for the block-store read path.
3. **Stop encoding for a metric.** `block_processor.rs:542` runs `to_proto` and a full encode to record block size. Use the received byte length, or `encoded_len()` (14 vs. 202 µs).
4. **Store what arrived.** For received blocks, the bytes on hand are already prost(`BlockMessageProto`); compress and store them instead of `to_proto`+encode. Keep re-encoding for locally created blocks.
5. **Remove transport round trips.** The send path encodes a `Packet`, caches the bytes, decodes them, then chunks (`packet_ops.rs:89–109`); gossip de-duplication re-encodes each (small) gossip `Protocol` to hash it (`grpc_transport_receiver.rs:476`).
6. **Move to a Merkle-root block hash in the next shard format** (Section 6). This removes the `to_proto` clone and the serial end-of-block encode, and adds per-deploy inclusion proofs. Until then, encode directly from the domain type (a hand-written `prost::Message::encode_raw`, pinned by golden tests) to drop the clone.
7. **Measure the real hot path.** Add a criterion bench and a replay flamegraph for RSpace's bincode-plus-Blake2b channel hashing. That is where serialisation runs per produce/consume, and its bytes are part of the shard format too (`models/build.rs:93` already canonicalises `locally_free` for it).
8. **Write down the canonical encoding, per shard format.** Shards currently depend on prost's behaviour (field order, `BTreeMap` maps, default-field elision). A short spec plus golden vectors, versioned with the shard format, protects deployments from a prost upgrade and lets partners implement compatible clients, whatever wire codec is chosen later.

9 Threats to validity

- Workloads are synthetic. Event counts per deploy, term shapes and deploy sizes were chosen to be plausible, not sampled from a live shard. Exporting a few real blocks from a soak run and re-running the bench would remove this caveat.
- The node was not built or profiled, so the budget in Table 5 combines microbenchmarks with call-path reading; domain conversions are estimated by struct clones.
- A 2-vCPU shared VM; absolute numbers will differ on validator hardware, ratios less so.
- The Cap'n Proto `Par` schema covers the constructors the generator emits, not all 58 `RhoTypes` messages; the full schema would not change the shape of the result.

A Reproducing the benchmark

The accompanying `serbench` crate contains `build.rs` (prost-build with the node's settings, capnpc), `schema/f1r3.capnp`, copies of the two `.proto` files from `dev`, and `src/main.rs`. It needs `protoc` and `capnp` on `PATH` and builds on stable Rust:

```
cargo build --release && ./target/release/serbench
```

It writes `results.csv` (nanoseconds) and `sizes.csv`; the numbers in this note are from those files.